From Zero to Your First Kotlin Multiplatform App

Building a Production-Grade Cross-Platform Mobile App with Compose Multiplatform & Clean Architecture

PRESENTER PROFILE

- **Speaker**
  Iqbal Fauzi (Senior Mobile Engineer at Bobobox)
- **Moderator**
  Bintang Poetra (Mobile Engineer at MaxxiTani)
- **Community**
  UZIRO KMP Developer Community (bit.ly/uziro-kmp)
- **Topic Focus**
  Kotlin 2.x, Compose Multiplatform 1.10, Room KMP, Ktor 3.3, Koin 4.1

WHAT YOU WILL LEARN

- **Modern Code-Sharing**
  Understanding the code-sharing spectrum from logic to full UI.
- **Clean Architecture & MVI**
  Structuring scalable multiplatform codebase with single source of truth.
- **Offline-First Strategy**
  Leveraging Room KMP and Ktor for resilient data synchronization.
- **Compose Multiplatform UI**
  Crafting responsive, native-performance UI for Android & iOS.

SPEAKER SCRIPT / NOTES:

Halo teman-teman! Selamat datang di sharing session UZIRO #01. Hari ini kita bakal kupas tuntas bagaimana membangun aplikasi mobile Android & iOS sekaligus menggunakan 100% Kotlin Multiplatform dan Compose Multiplatform lewat sample project Rick and Morty KMP.

Why Kotlin Multiplatform (KMP)?

A pragmatic approach to code sharing without runtime compromises

FLEXIBLE CODE-SHARING SPECTRUM

- **Logic-Only Sharing**
  Share Business Logic, Data, and Network layer while writing 100% native UI in Swift/SwiftUI and Jetpack Compose.
- **Full-Stack Multiplatform**
  Share 100% of the UI & Logic using Compose Multiplatform across Android & iOS.
- **Gradual Adoption**
  Can be adopted module by module into existing production apps without rewriting everything.

KMP vs OTHER FRAMEWORKS

- **True Native Compilation**
  Compiles directly to JVM bytecode on Android and native Apple Framework (LLVM/Objective-C interop) on iOS.
- **Zero Runtime Overhead**
  No heavy JavaScript bridge, webview container, or custom rendering engine overhead.
- **Single Tech Stack**
  Your mobile team writes Kotlin, uses standard coroutines, Flows, and familiar Android-like paradigms.

SPEAKER SCRIPT / NOTES:

KMP berbeda dengan hybrid engine lainnya karena tidak memaksakan runtime bridge. Kita bisa memilih porsi sharing sesuai kebutuhan: mau logic-nya aja, atau bablas sampai UI dengan Compose Multiplatform seperti di app Rick & Morty ini.

What We Built: Rick & Morty Multiverse Explorer

Production-grade features designed to demonstrate real-world patterns

CORE APP FEATURES

- **[Explorer] Multiverse Character Explorer**
  Search, dynamic filtering by status/gender/species, and infinite pagination.
- **[Locations] Dimension & Location Guide**
  Explore dimensions and characters residing in each location.
- **[Episodes] Interdimensional Episode Guide**
  List of episodes categorized by seasons with character associations.
- **[Offline] Favorites Vault (Offline-First)**
  Bookmark characters and episodes stored locally in cross-platform Room SQLite DB.

UX & DESIGN HIGHLIGHTS

- **[Design] Cyberpunk Sci-Fi Design**
  Material 3 tokens with PortalGreen, CyberYellow, and ElectricCyan glows.
- **[Adaptive] Edge-to-Edge Adaptive Layout**
  Clean window insets handling status bars and dynamic islands on iOS.
- **[Speed] High-Performance Image Loading**
  Integrated Coil 3 with Ktor engine for smooth avatar and asset caching.

SPEAKER SCRIPT / NOTES:

App ini bukan sekadar 'Hello World', tapi dirancang mendekati standar production: handle offline caching, complex filtering, type-safe navigation, hingga custom Material 3 theming yang adaptif.

Architecture Blueprint: Clean Architecture + MVI

Unidirectional Data Flow (UDF) for predictable state management

ARCHITECTURAL LAYERS

- **Presentation Layer**
  Compose Multiplatform UI observes immutable StateFlow from MVI ViewModel.
- **Domain Layer**
  Reusable UseCases encapsulating business rules and data transformations.
- **Data Layer**
  Repository pattern serving as Single Source of Truth orchestrating Local & Remote.
- **Data Sources**
  Remote API via Ktor Client & Local Cache via Room KMP Database.

MVI DATA FLOW CYCLE

- **1. User Intent / Action**
  User interacts with UI (e.g. OnSearchQuery, OnToggleFavorite).
- **2. ViewModel Execution**
  ViewModel receives intent and triggers domain UseCase coroutine flow.
- **3. Repository Resolution**
  Fetches from Room cache or Ktor HTTP endpoint.
- **4. State Emission**
  ViewModel updates immutable UI state, re-rendering Compose UI smoothly.

SPEAKER SCRIPT / NOTES:

Kita mengadopsi Clean Architecture + MVI. ViewModel mengekspos single immutable UI State ke Compose, sementara Use Cases mengisolasi business logic sehingga logic layer benar-benar agnostic dari platform.

The Powerhouse Multiplatform Libraries

Modern, stable multiplatform ecosystem powering Rick & Morty KMP

CORE LIBRARIES (PART 1)

- **Compose Multiplatform 1.10.x**
  100% shared declarative UI for Android & iOS.
- **Navigation Compose KMP**
  Official Jetpack Navigation with Kotlin Serializable type-safe routes.
- **Koin Multiplatform 4.1.x**
  Lightweight Dependency Injection with shared koinViewModel().
- **Kotlin Coroutines & Flow**
  Asynchronous pipelines and reactive UI state management.

CORE LIBRARIES (PART 2)

- **Ktor Client 3.3.x**
  Multiplatform HTTP engine (OkHttp on Android, Darwin on iOS).
- **Room Multiplatform 2.7.x**
  Official Google SQLite ORM for KMP with KSP code generation.
- **Kotlinx Serialization**
  Blazing fast JSON serialization for API & Navigation args.
- **Coil 3 Multiplatform**
  Modern async image loading with memory and disk caching.

SPEAKER SCRIPT / NOTES:

Dulu KMP minim library official, tapi sekarang ekosistemnya sudah sangat matang: Room sudah resmi multiplatform dari Google, Navigation Compose sudah multiplatform, Coil 3 dan Koin juga full support.

Under the Hood: Offline-First Data Layer

Writing shared SQLite database and networking code once in commonMain

ROOM MULTIPLATFORM STRATEGY

- **Shared Schema & DAOs**
  Entities and DAOs written once in commonMain with @Database annotations.
- **Platform Driver Injection**
  DatabaseBuilder created via Android context on Android and NSFileManager on iOS.
- **Continuous Offline Sync**
  Room DAOs expose Flow<List<Entity>> for automatic UI updates upon cache invalidation.

KTOR CLIENT & NETWORK RESILIENCE

- **Standardized HTTP Setup**
  Common Ktor HttpClient with ContentNegotiation, logging (Napier), and timeout config.
- **Network Result Wrapper**
  Custom Result<T> sealed interface catching network anomalies cleanly.
- **Repository Integration**
  Cache-then-network pattern: return local data instantly, sync latest from API in background.

SPEAKER SCRIPT / NOTES:

Salah satu highlight menarik adalah Room KMP. Kita bisa menulis Room Database dan DAO persis seperti di native Android, tapi sekarang berjalan langsung di iOS tanpa perlu bridging manual SQLDelight.

Compose Multiplatform & Type-Safe Navigation

Eliminating boilerplate and runtime navigation bugs across platforms

TYPE-SAFE NAVIGATION GRAPH

- **@Serializable Route Objects**
  Routes defined as Kotlin data classes instead of fragile string URL paths.
- **Type-Safe Argument Passing**
  Arguments (e.g. characterId) are automatically serialized and deserialized with full compile-time safety.
- **Bottom Navigation Sync**
  Centralized top-level destinations matching active route state.

ADAPTIVE DESIGN & THEME SYSTEM

- **Custom Design Tokens**
  Material 3 cyberpunk theme with sci-fi portal color palettes.
- **Window Insets & Edge-to-Edge**
  WindowInsets.safeDrawing guarantees content never clashes with status bars or camera cutouts.
- **Shared Component Library**
  Custom CharacterCard, FilterChips, and StatusBadge reused across all tabs.

SPEAKER SCRIPT / NOTES:

Dengan Jetpack Navigation Compose KMP, navigasi antar screen sekarang type-safe menggunakan serializable objects. Passing arguments jadi sangat clean dan aman dari runtime crash.

Practical Tips & Lessons Learned

How to avoid common pitfalls when starting your KMP journey

RECOMMENDED DO'S

- **Adopt Incrementally**
  Start by sharing Data/Domain layer before moving UI to Compose Multiplatform.
- **Use Gradle Version Catalogs**
  Centralize dependencies in libs.versions.toml for seamless version alignment.
- **Prefer DI over expect/actual**
  Use interfaces and Koin dependency injection rather than excessive expect/actual declarations.
- **Leverage Kotlin 2.0+ & K2**
  Take advantage of faster compilation and improved Multiplatform tooling.

CRITICAL DON'TS

- **Don't Ignore iOS Physical Testing**
  Always test scroll physics, font rendering, and gestures on real iOS devices or simulator.
- **Don't Leak Platform APIs**
  Keep domain Use Cases free from Android Context or iOS platform dependencies.
- **Don't Block Main Thread**
  Always dispatch database and network operations on Dispatchers.IO.

SPEAKER SCRIPT / NOTES:

Tips terbaik buat yang baru mulai: jangan over-engineer expect/actual. Cukup gunakan DI dan interface. Manfaatkan Version Catalog agar manajemen dependency tetap rapi.

Wrap-Up: Connect, Clone & Build!

Open-source resources to accelerate your Multiplatform mastery

REPOSITORIES & RESOURCES

- **[Repo] GitHub Repository**
  github.com/iqbalf/RickAndMortyKMP (Full source code, MIT license)
- **[Community] UZIRO KMP Community**
  Join Telegram & Discord community at bit.ly/uziro-kmp
- **[Speaker] Speaker Profile**
  github.com/iqbalf (Senior Mobile Engineer @ Bobobox)
- **[Host] Moderator Profile**
  github.com/bintangpoetra (Mobile Engineer @ MaxxiTani)

NEXT STEPS FOR YOU

- **1. Clone the project**
  Run ./gradlew :androidApp:installDebug or open in Xcode.
- **2. Explore the commonMain module**
  Inspect how Room, Ktor, and Koin are configured in one place.
- **3. Try adding a new feature**
  Add episode search or location bookmarking using the existing pattern!
- **4. Q & A Session**
  Feel free to ask any questions about KMP setup, architecture, or lessons learned!

SPEAKER SCRIPT / NOTES:

Repo-nya sudah open-source dan siap di-clone untuk bahan belajar atau starter template teman-teman. Sekarang kita buka sesi tanya jawab!